

多维数组

$a[n][m]$ 形式为: $a = \begin{bmatrix} a[0][0] & a[0][1] & a[0][2] & \dots & a[0][m-1] \\ a[1][0] & a[1][1] & a[1][2] & \dots & a[1][m-1] \\ \dots & \dots & \dots & \dots & \dots \\ a[n-1][0] & a[n-1][1] & a[n-1][2] & \dots & a[n-1][m-1] \end{bmatrix}$

存储时按一维数组, 顺序 $a[0][0], a[0][1], \dots, a[0][m-1], a[1][0], \dots, a[1][m-1], \dots$

若 $a[0][0]$ 的地址为 α , 则 $a[j][k]$ 的地址为 $\alpha + (j \times m + k) \cdot \text{sizeof}(T)$

高维数组同理, 用递推公式易得。

• 特殊的矩阵及其处理

(一) 对称矩阵: 有 $n \times n$ 方阵 A , 对 $\forall i, j$, 满足 $a_{ij} = a_{ji}$

存储时只存对角线及以上的元素(上三角矩阵), 或下三角矩阵。

这样存储可以把原来需要 n^2 的空间节约到了 $\frac{n(n+1)}{2}$ 。

• 一般采用行优先的方式存储, 即只存下三角部分, 用于存储 $a[n][n]$ 的数组 B 如下:

$a_{0,0}$	$a_{1,0}$	$a_{1,1}$	$a_{2,0}$	$a_{2,1}$	$a_{2,2}$	$\dots$	$a_{n-1,0}$	$a_{n-1,1}$	$\dots$	$a_{n-1,n-1}$
-----------	-----------	-----------	-----------	-----------	-----------	---------	-------------	-------------	---------	---------------

故 a_{ij} 在 B 中存放的位置为 $\text{Loc}(i, j) = 1 + 2 + \dots + i + j = \frac{i(i+1)}{2} + j$ ($i \geq j$ 时)

若 $i < j$, 则 $a_{ij} = a_{ji}$, 故 $\text{Loc}(i, j) = \frac{j(j+1)}{2} + i$ (即 i, j 互换)

• 列优先存储同理, 但是存上三角方式, 第 $i-1$ 行存 $n-i+1$ 个元素。

$\text{Loc}(i, j) = n + (n-1) + \dots + (n-i+1) + (j-i) = \frac{1(2n-i-1)}{2} + j$ ($i \leq j$ 时)

若 $i > j$, 则 $a_{ij} = a_{ji}$, 故 $\text{Loc}(i, j) = \frac{j(2n-j-1)}{2} + 1$ (即 i, j 互换)

(二) 三对角矩阵: 只有主对角线及其左右两条对角线上的元素不为 0 的稀疏矩阵

用数组 B :

$a_{1,1}$	$a_{1,2}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	$a_{3,2}$	$a_{3,3}$	$a_{3,4}$	$\dots$	$a_{n,n-1}$	$a_{n,n}$
-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	---------	-------------	-----------

B 共有 $3n-2$ 个元素, 设 $a_{i,j}$ ($1 \leq i \leq n, i-1 \leq j \leq i+1$) 在 B 中位置为 $2i+j-3$

(三) 稀疏矩阵: 非零元素占矩阵总元素的比小于 0.05 的矩阵即可认为是稀疏矩阵。

存储方式: 用三元组 (i, j, a_{ij}) 唯一确定 A 的一个非零元素, 用三元组数组存储整个 A

三元组表按行优先的顺序存储, eg:

0	1	0	1
0	0	0	0
0	0	2	0
1	0	0	0

 $\rightarrow$

row	col	value
0	0	1
0	2	1
0	3	1
1	2	2
2	3	1

• 矩阵的转置:

① 简单算法: 对每一个列数 col , 扫描三元组表, 若有则将其行列交换存入新表。

② 快速算法: 对三元组表中的每一个元素, 提前确定其在新表中的位置, 然后边扫边填。

辅助数组: $rowSize$ 数组: 矩阵 A 各列非零元素个数(扫三元组即得)

$rowStart$ 数组: 矩阵 B 各行的元素在 B 中开始存放位置(存一个就+1)

eg: 扫表得 $rowSize$:

$[0]$	$[1]$	$[2]$	$[3]$	$[4]$	$[5]$	$[6]$
1	1	1	2	0	2	1

 (即 B 每行有几个待填)

$\rightarrow rowStart[i] = rowStart[i-1] + rowSize[i-1]$ 计算 $rowStart$ 数组;

$[0]$	$[1]$	$[2]$	$[3]$	$[4]$	$[5]$	$[6]$
0	1	2	3	5	5	7

那么再扫三元组, 扫到列号为 i , 就行列交换后存在 $B[rowStart[i]]$ 的位置,

然后 $rowStart[i]++$ (先扫到的在 B 中列号在前, 因此优先)

• 十字链表法存稀疏矩阵

表头结点:

head	next
down	right

 $\rightarrow$ 非零元素结点:

head	row	col
down	value	right

 $\rightarrow$

相当于每行、每列都是一个单独的循环链表。

(四) 字符串: n 个字符组成的有限序列 $S = "a_0 a_1 a_2 \dots a_{n-1}"$

串的长度 n 即为 S 中 a_i 的个数, 不包含作为分界符的引号和串结束符 $'\backslash 0'$

空串: 长度为 0 的串; 空白串: 除 $'\backslash 0'$ 外, 包含的其他字符均为空格的串

• $string.h$ 库函数: 可以直接用、查书

存储方式: (静态/动态) 数组、链表

结构体存好当前大小和最大空间, 满后成倍扩充空间

串的模式匹配: 在主串中寻找子串在串中的位置

eg: 目标 T : Nanjing, 模式 P : jin $\rightarrow$ 匹配结果: 3 (即位置)

• 朴素算法: 遍历 T (从 $T[0]$ 到 $T[m-n]$), 对每一个 $T[i]$ 考虑若 $T[i]$ 到 $T[i+n-1]$

恰分别等于 $P[0]$ 到 $P[n-1]$, 即输出当前 i (时间复杂度 $O(m \times n)$)

• KMP 算法:

先考虑模式串 P 对应的 $next[]$ 数组, 规则见课本 (即后缀最长 = 前缀的 k)

eg:

P	a	b	a	b	d	a	b	a	b	a	b
$next(j)$	-1	0	0	1	2	0	1	2	3	4	3

获取 $next$ 数组代码: `void getNext(int next[]) { int j=0, k=-1;`

`while (j < length_P) {`

`if (k == -1 || ch[j] == ch[k]) { j++; k++; next[j] = k; }`

`else k = next[k]; } }`

然后母串指针从 0 开始, 与 P 对应的指针位置按位比较

• 若匹配, 则 K, j 指针均++ 接着比, 否则考虑 $next(j)$ 的取值:

$\begin{cases} \text{为 } -1: \text{母串指针 } k++, P \text{ 的指针 } j \text{ 归 } 0 \text{ 接着比。} \\ \text{否则: 母串指针 } k \text{ 不变, } P \text{ 的指针 } j \text{ 取 } next(j) \text{ 的值接着比。} \end{cases}$

• 因避免了 k 指针的回退, 时间复杂度为 $O(m+n)$, 远优于 $O(m \times n)$ 。

(五) 广义表: n 个表元素 (可以是子表, 或数据元素) 组成的有限序列, 记作 $LS(a_1, a_2, \dots, a_n)$

若 $n \geq 1$, 则定义第 1 个表元素 a_0 为表头, 除 a_0 外其他元素组成的表为表尾

性质: 有次序性、有长度、有深度 (表的总深度为其深度最大的子表的深度+1)、可递归、可共享

* 广义表的表示: 每个表结点由 3 个域组成:

utype	info	tlink
-------	------	-------

$utype \begin{cases} 0: \text{表头} \\ 1: \text{原子数据} \\ 2: \text{子表} \end{cases}$ $info \begin{cases} utype=0 \text{ 时, 存引用计数 ref (被引用次数)} \\ utype=1 \text{ 时, 存相应的数据值 (value)} \\ utype=2 \text{ 时, 存指向子表头的指针 (link)} \end{cases}$ $tlink \begin{cases} utype=0 \text{ 时, 指向第 } i \text{ 个结点} \\ utype \neq 0 \text{ 时, 指向下一结点} \end{cases}$

广义表的递归算法: 复制; 求最大长度、深度; 判断是否相等; 删除; 建立